ChatGPT

amiga_gui_research_3x.md

Part A – Core OS Contracts (Workbench 3.0/3.1)

Intuition & Windowing (Workbench Screen)
You may assume: - The Workbench screen is a public screen that Intuition opens at boot and keeps open for sharing [1]. Applications can safely open windows on the Workbench screen by specifying `WA_PubScreenName, "Workbench"` (or using the default public screen lock) – Intuition will clip graphics and manage overlapping windows for you [2]. Only one window is "active" at a time and receives input (keyboard/mouse) focus [3], but multiple windows can coexist and update independently. Intuition delivers input events (mouse moves, clicks, key presses, etc.) as messages to your window's IDCMP message port for any event types you subscribe to [4]. Standard window UI elements (drag bar, close gadget, depth arrangement) and behaviors (like front-to-back stacking and user resizing) are handled by Intuition automatically.

- An Intuition window opened with the proper flags (e.g. `WA_CloseGadget, TRUE`, `WA_DragBar, TRUE`, etc.) will have the expected GUI elements provided by the OS. For example, if you include `IDCMP_CLOSEWINDOW` in `WA_IDCMP`, clicking the window's close gadget will cause Intuition to send an `IDCMP_CLOSEWINDOW` message to your window's port [5] [6]. Intuition handles the low-level details (like rendering gadgets and queuing messages) so your program can focus on handling these events. You can rely on the OS to manage window layering, screen redraws when windows move or overlap, and to restore obscured portions of your window when it becomes visible (on a `SMART_REFRESH` or `SimpleRefresh` window).

- Intuition **requires** that you clean up resources: when your program exits, it must close any windows (and screens) it opened. The OS will not automatically close a window left open by a terminated process – it will remain orphaned [7]. Thus, a well-behaved program always calls `CloseWindow()` (and other cleanup) before exit. You can assume that closing an Intuition window via `CloseWindow()` will free its internal resources (Intuition will reclaim memory for the Window structure, Layer, etc.) – but **only** if you have removed/detached any menus or custom gadgets first (see below).

You must not assume: - That Intuition will "manage" or auto-reply to your IDCMP messages. Your code is responsible for obtaining (`GetMsg()`) and replying (`ReplyMsg()`) each `IntuiMessage` exactly once [8] [9]. Failing to reply an IDCMP message prevents Intuition from cleaning up resources associated with that message (e.g. memory or signal bits), which can lead to memory leaks or hung input. Always perform a loop to get and reply all pending messages after Wait(). Similarly, do **not** close a window while you still have un-replied messages for it; process and reply them (or use a safe break-out pattern) before calling `CloseWindow()`. Closing a window out-of-sequence (e.g. from within the message loop that is iterating its own messages) can lead to reading a freed message port on the next `GetMsg()` [10].

- That you can modify or inspect Intuition's data structures arbitrarily. Many fields in `struct Window` and `struct IntuiMessage` are for OS internal use. For example, the `IAddress` field of an IntuiMessage may point to different things depending on message Class – it could be a `struct Gadget *` for gadget events or something else entirely [11]. **Never** cast or

dereference `IAddress` unless you know the message Class and what it contains. In general, treat Intuition pointers (Window, Screen, Gadget, etc.) as opaque handles and use the documented APIs to query or modify state. Also, do not assume a pointer remains valid after the associated object is closed or freed – e.g., a `Window->RPort` (RastPort) pointer is only valid while the window is open.

- That one Window's IDCMP can be freely reused for another without precautions. If you share a single `MsgPort` for multiple windows (an advanced technique), be very careful: calling `ModifyIDCMP(window,NULL)` improperly or closing one window out of several sharing a port can confuse Intuition [10]. The simpler, safe assumption is each window has its own UserPort (the default when you let `OpenWindowTags()` allocate it). If you do use a custom shared port, ensure you remove any window's IDCMP hooks and messages from it before closing that window.

- Any undefined behavior or "quirk" will remain consistent. For instance, do not assume that the Workbench screen will always use a particular default font or color – query `Screen->Font` or `GetScreenAttr()` if needed, rather than hard-coding assumptions. Also, **do not** use Exec's `Exit()` to terminate a Workbench-launched program – doing so bypasses C cleanup and can prevent the program's code from unloading under Workbench [12]. Always return from `main()` or call `exit()`, so the C runtime can free resources and Workbench can unload the module.

GadTools Library (VisualInfo, Gadgets)
You may assume: - GadTools provides a higher-level API to create standard GUI widgets ("gadgets") like buttons, string fields, checkboxes, lists, etc., with a consistent Look&Feel introduced in V36+ ("New Look" GUI). Before creating any GadTools gadgets or menus, you must obtain a **VisualInfo** handle for the target screen using `GetVisualInfo()` [13] [14]. This VisualInfo carries screen-specific rendering info (font metrics, default 3D frame colors from the screen's DrawInfo, etc.) so that GadTools can draw gadgets correctly. You may assume that as long as you supply the correct VisualInfo (via `ng.ng_VisualInfo` in each NewGadget or as an argument to `LayoutMenus()`), GadTools will render with the proper 3D style and use the screen's colors and font. The VisualInfo remains valid until you free it, and should be freed with `FreeVisualInfo()` when you are done (after freeing gadgets/menus) [15] [16].

- GadTools gadgets are created in a two-step process: first call `CreateContext()` to initialize an empty gadget list, then call `CreateGadget()` repeatedly to add gadgets [17] [18]. Each gadget type (kind) has its own set of tags. For example, a `BUTTON_KIND` gadget might use `GA_Text` for its label; a `STRING_KIND` uses `GTST_String` for initial text; a `LISTVIEW_KIND` requires `GTLV_Labels` to point to a list of items, etc. You can assume the GadTools autodocs are correct regarding which tags are required for each kind. GadTools will link your gadgets into an Intuition-compatible list automatically. It even allocates multiple Intuition gadgets for certain composite GadTools gadgets (for example, a ListView may internally create a ListView area and a vertical scroll bar gadget) – this linking is handled for you as long as you pass the previous gadget pointer returned by `CreateGadget()` in each subsequent call [19] [20].

- Each GadTools gadget, once created and attached to a window (by specifying `WA_Gadgets` when opening the window, or via `AddGList()`), will generate Intuition IDCMP events in the standard way. For instance, when a button or checkbox is clicked, your window receives an `IDCMP_GADGETUP` message. When a string gadget is active and the user hits Enter (or Tab/Shift-Tab for navigation), you get an `IDCMP_GADGETUP` with specific code (0x09 for tab, etc.) [21]. The OS guarantees certain behaviors: string/integer gadgets only emit an event when the

user explicitly ends input (Enter/Tab/etc.) [22], and cycle gadgets emit `IDCMP_GADGETUP` when the choice changes. GadTools gadgets honor standard gadget flags like `GA_Disabled` (to grey them out) and will update their visuals accordingly if you set those attributes at creation or via `GT_SetGadgetAttrs()`.

- GadTools relies on some global data (the `IntuitionBase` or interface) and requires that the `gadtools.library` be open (which normally is handled by `-lauto` or opened for you if you include <proto/gadtools.h>). If using VBCC or another modern compiler with `-lauto` or auto-open stubs, you can assume `GadToolsBase` is opened for you. Otherwise, you must open it before calling GadTools functions. (VBCC with `-lauto` automatically opens `intuition.library`, `gadtools.library`, etc., which makes the code identical to SAS/C code that assumed libraries open via pragmas.)

You must not assume: - That you can bypass **VisualInfo** or **DrawInfo** requirements. If a GadTools function or tag is documented to need a VisualInfo, you must provide it; otherwise the appearance or sizing of gadgets may be wrong or the function may fail. For example, GadTools `CreateMenus()` requires a VisualInfo pointer for proper layout on v39+ (New Look menus). Similarly, do not assume a VisualInfo from one screen can be used on another – always get a new VisualInfo for the screen your window is on. If your program opens its window on the Workbench screen, lock the public screen and call `GetVisualInfo()` on that screen [23]. **Do not** free the VisualInfo until *after* you have closed the window and freed all gadgets and menus that used it [15].

- That you can directly manipulate GadTools gadget structures or internal flags. The GadTools gadgets are built on Intuition's `struct Gadget`, but you must treat them as managed by the system. For instance, never write directly into a `StringInfo->Buffer` for a string gadget – use `GTST_String` tag to set initial text, or read the buffer via the provided pointer if needed [24]. Do not toggle gadget->Flags or other fields by hand; use `GT_SetGadgetAttrs()` to change attributes at runtime. The **only** exception is a documented trick: setting `GA_Immediate` on a string gadget *at creation time* is allowed (the GadTools manual explicitly notes that passing `{GA_Immediate, TRUE}` to `CreateGadgetA()` is supported to force immediate feedback) [25]. Aside from such sanctioned cases, modifying GadTools gadgets directly can break compatibility and is undefined [26].

- That the gadget state in memory is static or updated synchronously with events. For example, reading a checkbox gadget's `GFLG_SELECTED` flag is a valid way to get its state (since GadTools doesn't provide a separate boolean), but note that the gadget struct might already be updated by the time you process the IDCMP message. If the user clicks twice very quickly, the `GFLG_SELECTED` bit may have toggled again by the time you inspect it [27]. Intuition does *not* queue every intermediate state change. The safe pattern is to trust the IDCMP event to tell you something changed, then query the gadget's state if needed (or maintain your own copy of the state). In summary, **do not assume** gadget internal fields are locked during your event handling; always design for the possibility that dynamic fields (like a gadget's Selected state) could change without additional messages if the user interacts again rapidly.

- That GadTools will copy your data structures. Notably, for a ListView gadget, the `GTLV_Labels` tag expects a pointer to an **Exec** `struct List` whose nodes' ln_Name fields are the item strings [28]. GadTools does *not* duplicate the list; it will traverse it directly to display items. Thus, **you must not** free or alter that list while the gadget is using it. If you need to modify the list (add/remove items, change text), you must first "detach" it from the gadget by calling `GT_SetGadgetAttrs()` with `GTLV_Labels, ~0` (a special value to detach) [29]. This

prevents GadTools from iterating the list on another task while you modify it. After updating the list, reattach it with `GTLV_Labels` pointing to the new list head [30]. Failing to follow this protocol can cause crashes or random behavior (because GadTools might traverse your list on a separate input handler task while you change it). In summary, do not assume the list is copied internally – manage it carefully and always detach before changes. Similarly, for a Cycle gadget, if you use `GTCY_Labels` with an array of strings, those strings must remain valid in memory for the life of the gadget (GadTools doesn't copy the text) [31].

- That dynamic resources are freed automatically when a window closes. GadTools gadgets created with `CreateGadget()` are not freed by Intuition when the window is closed – they remain in memory until you call `FreeGadgets()`. The proper order is to close your window first (which detaches the gadgets from Intuition), then call `FreeGadgets(gadgetList)` to free all gadgets in the list. The same goes for menus created with GadTools: remove them from the window (via `ClearMenuStrip()` or by closing the window) and then call `FreeMenus()` to deallocate them. Only after freeing gadgets/menus should you free the VisualInfo and unlock the screen [15] [32]. *Never* free a gadget or menu while it is still active/attached to an open window [33] – that is undefined behavior and will likely crash.

Menu Handling (NewLook Menus via `NewMenu`)
You may assume: - Workbench 3.x uses the "New Look" 3D menu system (introduced in 2.0). Programs can create menus using the GadTools menu functions and the `NewMenu` structure. A menu bar is described as an array of `struct NewMenu`, which the system converts into Intuition Menu and MenuItem structures. The OS provides `CreateMenus()` to allocate and build the menu list from your `NewMenu` array, and `LayoutMenus()` to compute sizes/positions (this two-step is required) [34]. You should assume that after calling `CreateMenus(newMenuArray, TAG_END)` you will get a non-NULL `Menu *` if allocation succeeded; then you **must** call `LayoutMenus(menuStrip, visualInfo, GTMN_NewLookMenus, TRUE, TAG_END)` to layout the menus with the NewLook 3D style [34]. After these calls, the menu strip can be attached to your window with `SetMenuStrip(window, menuStrip)`. Intuition will then display that menu bar whenever your window is active. The layout step calculates item widths, positions, shortcut underlines, etc., which *are not done* by CreateMenus alone. (It's safe to assume `LayoutMenus()` will use the VisualInfo you pass to apply the current screen's font and styling.)

- Only one menu bar is visible per screen: whichever window is active on the Workbench screen controls the shared menu bar at the top [35] [36]. Thus, when your window becomes active, its menu strip is shown; when inactive, the menu bar is replaced by the other program's menu (or none, if that program has no menu). Intuition handles this automatically when you use SetMenuStrip/ClearMenuStrip. You may assume that selecting a menu item (by holding the right mouse button and clicking an item) will generate an IDCMP message to your window (`IDCMP_MENUPICK`). The message's `Code` contains a menu identifier in which the high bytes encode the menu and item number. Use Intuition helpers or macros to decode it (e.g. the `ItemAddress()` function can retrieve a pointer to the `MenuItem` from the code). If multiple menu items are selected at once (e.g. checkmarked items or multi-select menus), Intuition will send the first selection and set the `NextSelect` field of the MenuItem for you to iterate; you loop calling `ItemAddress()` and then get `item->NextSelect` until it equals `MENUNULL`. This is standard and expected; assume that you need to handle the possibility of multiple selected items from one menu pick event.

- Menu items can have keyboard shortcuts (qualifiers). The `NewMenu` structure allows an item to have a key equivalent (e.g. "O" for Open, with an Amiga-key or Ctrl chord) by supplying the letter

in the `nm_CommKey` field. You may assume these are processed by Intuition: if the user presses the corresponding key combination, your program will receive an IDCMP_MENUPICK just as if the user pulled down the menu and selected the item. Also, `nm_Flags` can mark items as checked or disabled initially (e.g. NM_ITEM with `NMIF_CHECKED` will appear ticked). The NewMenu->nm_MutualExclude field is used to group exclusive items (radio menu behavior) by sharing a non-zero mask. Intuition will enforce mutual exclusivity within that mask group. All these behaviors are part of the OS contract for menus and do not require manual intervention beyond setting up the NewMenu flags properly.

- Each `NewMenu` has a user data field (`nm_UserData`). You can use this to store an identifier or pointer for your own use [37]. Intuition/GadTools does not touch `nm_UserData` except to copy it into the corresponding Menu or MenuItem's `MenuItem.mi_UserData` field. You may assume that after menu creation, you can retrieve this via `ItemAddress()` and then check the MenuItem's `mi_UserData`. Many developers store a constant or enum value here to identify which menu option was picked, instead of decoding menu numbers. This is a safe and recommended practice on 3.0/3.1. For example, you might set `nm_UserData = (APTR)MY_CMD_QUIT` for the "Quit" menu item; then in your IDCMP handler, find the selected item and switch on that user data pointer. This simplifies menu handling and is fully supported by OS3.x [37].

You must not assume: - That *CreateMenus()* alone is enough. As noted, **always** follow it with `LayoutMenus()` [34]. If you skip LayoutMenus, your menus will not have proper dimensions and may not display correctly (in fact, Intuition may refuse to render the menu strip). This is a common mistake; the contract is that you call both in sequence and check for errors (both return BOOL success). Also, you should not assume a menu strip is automatically active once set – your window must be active for the user to see it. Intuition will automatically refresh the menu bar when windows activate or their menu set changes, so you don't normally need to force a redraw except after enabling/disabling items (use `OnMenu()` / `OffMenu()` for that).

- That you can alter menu structures on the fly without updating Intuition. If you want to change menu items (add or remove menus, change labels) while the menu is attached, you cannot just poke the `Menu` structure. You *must* first detach the menu (`ClearMenuStrip()`), then either free and rebuild it, or use `ResetMenuStrip()` after making minor changes, followed by `LayoutMenus()` again if needed. In 3.0/3.1, there is limited support for dynamic menu changes; the simplest, safe approach is to remove and re-create the menu strip. Do not assume Intuition will notice changes to menu item text or flags on its own. And **never** free a `Menu` or `MenuItem` structure that Intuition is currently using; always ClearMenuStrip first, then FreeMenus.

- That all documented features are bug-free in 3.0/3.1. Notably, using BOOPSI imagery in menus (NewMenu `nm_Type = NM_ITEM` with `nm_Label` pointing to a BOOPSI Image object) was supposed to be supported in V39+, but in practice on Kickstart 3.1 it does **not** work and can crash `LayoutMenus()` [38] [39]. The official autodoc claimed OS3.0+ allows BOOPSI images in menus, but the reality is contradictory: the gadtools include file explicitly says only classic Intuition images can be used [40]. **Do not assume** you can safely use `IM_ITEM` or `IM_SUB` with custom image classes on 3.0/3.1. The safe contract is to use text labels and simple checkmarks (Intuition adds checkmark images internally for `CHECKED` items) on these OS versions. (This bug was not resolved until much later; the workaround is to avoid image menu items or use only traditional `struct Image` data for menu imagery.)

- That Workbench/Intuition will automatically enforce menu usage rules beyond the basics. For example, Intuition will prevent two menu items with the same mutual-exclude mask from being simultaneously selected, but it won't stop you from enabling a "Preferences..." menu item while a modal prefs window is already open (that logic is up to the application). Also, do not assume the menu shortcuts you assign won't conflict with Workbench's own keys or other programs – Workbench screen menu shortcuts (like *Workbench* menu) take priority when Workbench is active, but when your app is active, your shortcuts have effect. It's on the developer to choose unique shortcuts to avoid confusion.

ASL Requesters (File/Font Requesters)

You may assume: - The `asl.library` provides standard file and font requesters that open in a modal sub-window and return user-selected file paths or font names. To use ASL, you allocate a requester object (FileRequester or FontRequester) by calling `AllocAslRequest()` (or the convenience `AllocAslRequestTags()`), then invoke it with `AslRequest()` [41] [42]. On OS3.0/3.1, two types are available: `ASL_FileRequest` and `ASL_FontRequest` (screen mode requesters were introduced in later versions, not in 3.1). You should assume that `AllocAslRequest(ASL_FileRequest, ...)` returns a pointer to an internal **FileRequester** structure, which contains fields like `fr_Drawer` (selected directory path) and `fr_File` (selected file name). After calling `AslRequest()`, if it returns TRUE (user picked something), those fields will be filled with the user's selection [43]. If it returns FALSE, the user canceled or an error occurred, and the contents remain as before. The ASL requester is synchronous – your code will block until the requester is closed by the user [43]. This means you don't need a separate event loop for the requester; when `AslRequest()` returns, the requester window is already closed.

- The requester's data persists across multiple invocations. If you call `AslRequest()` on the same allocated requester a second time, it will "remember" the last state – e.g., the last directory navigated, the last typed filename, etc., unless you override them with new tags [44]. This behavior allows implementing an "Open" dialog that opens in the last used folder on subsequent uses, for instance. It also means you can set certain tags once at allocation (like an initial drawer, or title) and they remain in effect, while other tags can be passed each time to change behavior per invocation. This is by design in OS3.x. For example, you might allocate a FileRequester once at program startup with an initial drawer and pattern, then call AslRequest whenever needed without re-allocating each time. (Make sure to free it on exit.)

- ASL can be attached to your window or screen. By default, if you do nothing, `AslRequest()` will open its requester on the Workbench screen (or the screen of the window specified by `WBENCHSCREEN` by default). However, you can ensure it opens on the same screen as your GUI by using the `ASL_Window` tag to attach it to one of your windows [45]. If you provide `ASL_Window, myWindow`, the file requester will open centered on that window's screen (and usually will share that window's IDCMP port unless you also request a new IDCMP with `FILF_NEWIDCMP` flag) [46]. In practice, sharing the IDCMP doesn't require extra handling because `AslRequest()` is synchronous – it manages the messages internally and returns control after the requester is closed. You can assume that attaching to a window also keeps the requester on top and modal relative to that window (user can't interact with your window behind it until they choose a file or cancel). This is the typical usage for a GUI "Browse..." button: it opens an ASL requester attached to your main window.

- After a successful `AslRequest()` call, the selected path/name can be retrieved from the requester structure. In OS3.x, the FileRequester has fields like `fr_Drawer` (C string of the drawer path, e.g. "DH0:Projects") and `fr_File` (filename, e.g. "data.txt"). You typically

concatenate them (with a separator if needed) to get the full path. The ASL requester also provides `fr_LeftEdge`, `fr_TopEdge`, etc., if you need to know where the requester was on screen, and flags like `fr_DrawerOk` to indicate if a drawer was chosen. These details are documented in the RKMs. In fonts requesters, similarly, you get the chosen font name and size. The key guarantee is that these fields are valid until you free the requester.

You must not assume: - That you can use the ASL requester without properly allocating it. **Always** call `AllocAslRequest()` (or the older `AllocFileRequest()` for V36) to get a new requester. The structure it returns is managed by asl.library. Do not bypass this by declaring a FileRequester structure in your own code – the library may initialize additional data beyond the visible struct fields. The OS specifically warns: *"Do not create the data structure yourself. The values in this structure are for read access only. Any changes to them must be performed only through asl.library calls."* [47]. So, you must not write into the FileRequester's fields directly; for example, to set a default path, use the `ASL_Dir` or `ASL_Drawer` tag at allocation or request time, rather than trying to strcpy into `fr_Drawer`. The only "write" you do is via tags. Treat the structure as read-only once returned.

- That the memory from an ASL requester is freed automatically. You are responsible for calling `FreeAslRequest(requester)` when you no longer need it [48]. Failing to free it will cause memory leaks (the internal buffers and state will remain allocated). FreeAslRequest will deallocate the internal data and the structure itself. As a rule, pair each successful AllocAslRequest with a FreeAslRequest when done. (It is safe to FreeAslRequest at any time; if a requester window was open at that moment – which can't happen if you're calling it after AslRequest returns – it would handle closure internally first. But since AslRequest is synchronous, just free after use or at program exit.)

- That addresses provided to ASL remain valid forever. If you pass in a pointer via tags, such as a custom title string (`ASL_Hail`) or a drawer name in a buffer, the ASL requester may keep a reference to it internally between calls [44]. It will not copy your custom strings unless documented. For example, if you use `ASL_Pattern` with a pointer to a pattern string, you should keep that pattern string allocated until you free the requester (or pass a new one next time). Also, after a call, if you plan to use the results *after* freeing the requester, you must copy them out to your own buffers. Once you call FreeAslRequest, the `fr_File` and `fr_Drawer` pointers become invalid. So do not assume you can free the requester and still use the file name string it last returned – instead, strdup it or copy to a safe buffer *before* freeing.

- That ASL requesters handle multithreading or reentrancy. They are designed to be called from a GUI task context. Do not attempt to open multiple ASL requesters simultaneously from one task (the library isn't reentrant in that way on classic OS) – open them one at a time. Also, ensure your program has sufficient stack when using ASL; these routines can be stack-heavy (especially font requester or if a directory has many entries). Workbench typically launches programs with a default stack (usually 4K or as set in the icon ToolType). If your GUI uses ASL heavily, consider recommending a larger stack via the icon. This isn't an OS guarantee, just a practical caution.

- Any behavior beyond documented OS3.1 features. For instance, ASL file requester in 3.0/3.1 does **not** support multiple file selection – one invocation returns one file. It also does not offer separate "drawing" of icons or additional file info like later version requesters. Don't assume features that came in OS3.5+ (like ASL screen mode requester, multi-select, recent drawers lists) exist – they do not in 3.0/3.1. Keep to the basics: file/drawer selection and font selection.

Workbench Integration (Workbench Screen & Tools)

You may assume: - When opening a window on the **Workbench screen**, that screen will remain open and available as long as Workbench itself is running (which is normally for the duration of your program). The Workbench screen is a public screen named "Workbench". You can lock it via `LockPubScreen("Workbench")` (or `LockPubScreen(NULL)` to lock the default public screen) to obtain a `struct Screen*` pointer for it [23]. This ensures the screen won't close (in rare cases, if Workbench were to exit) while you prepare GUI elements. It's common to lock the screen, get VisualInfo, create gadgets/menus, then open the window with `WA_PubScreen` or `WA_CustomScreen` pointing to that Screen. After closing your window, you'd `UnlockPubScreen(NULL, screen)` to release it [32]. If you instead use `WA_PubScreenName, "Workbench"`, Intuition will handle locking/unlocking automatically for that window – you typically don't have to manually LockPubScreen in that case, unless you need the screen pointer for VisualInfo before opening the window. The OS contract here is that the Workbench screen can be shared by any number of program windows [49], and that it won't vanish out from under your program. (Only an expert user could manually quit Workbench, and even then your locked screen pointer prevents it until unlocked.)

- If your program is launched from Workbench (by double-clicking an icon), the OS will pass your process a `WBStartup` message containing any icon ToolTypes or project file parameters. In C, if you use the standard startup code, `argc==0` indicates a Workbench launch. The startup code (in clib or VBCC's `-lauto`) will have already called `GetMsg()` on your proc's message port to remove the WBStartup message [50], so you don't usually have to deal with it manually in high-level languages. You may assume that if `argc>0` you have a CLI launch, and if `argc==0` (or `argv[0]` is NULL depending on compiler) you have a Workbench launch with no CLI stdout/stderr available. Workbench does not automatically open a console for programs (unless an icon has a custom tooltype like `CON:`). Therefore, printing to stdout when started from WB is a no-op at best, or a crash at worst if no handler is attached [51]. The safe assumption is Workbench-launched GUI programs should not use console I/O (or should open their own console window if needed). This separation of CLI vs WB context is guaranteed by the OS – two different ways to start an app.

- A well-behaved Workbench GUI application should adhere to certain conventions that OS3.0/3.1 enforce or encourage. For example, if your program has an AppIcon or AppWindow (icons you drag-and-drop onto), those use `workbench.library` functions – but for normal tools that just open a window, the main integration point is the icon (.info file) that provides ToolTypes and default stack, etc. The OS guarantees that if the user double-clicks your app's icon, Workbench will launch it with no CLI and your code can read ToolTypes via `GetVar()` or similar DOS calls (since the WBStartup message includes a lock to the icon's directory and a table of ToolType strings). You may assume Workbench will stay responsive even while your window is open – Workbench itself is just another task with its own window(s). As long as you handle input events and don't block the system in tight loops without yielding, Workbench and your app will multitask cooperatively (preemption in Exec ensures this). In short, integration means coexisting nicely on the Workbench screen and following system rules (which your program will if you use Intuition and GadTools calls properly).

You must not assume: - That launching from Workbench is identical to CLI. In particular, **do not** write to `stdout` or `stderr` in a Workbench-launched program unless you've explicitly opened a console or log file. If you call `printf()` when `argc==0` (no console), the C library might attempt to write to a null or invalid file handle, causing a crash [51]. The AmigaOS rule: Workbench programs should either refrain from console output or open their own output window (e.g. via `Open()` on "CON:"). Many developers include a runtime check like:

```
if (argc == 0) {
    // Workbench launch, open a console for output if needed
    Open("CON:0/0/640/200=Output", MODE_OLDFILE);
}
```

or simply avoid all printf logging unless debugging. Similarly, do not assume you can obtain command-line arguments when launched from an icon – you must parse ToolTypes instead.

- That Workbench provided resources can be freely manipulated. For example, the WBStartup message provides your program with a lock on its icon's drawer (and possibly locks for any project files icons double-clicked onto it). **Do not** call `UnLock()` on those locks unless you duplicated them with `DupLock()`. They belong to the system. The common bug is calling `UnLock()` on `WBStartup->sm_ArgList[i].wa_Lock`, which can corrupt Workbench's lock count and lead to crashes on exit [52]. The correct approach is to use those locks as needed (e.g. CurrentDir() to the program's directory) but not manually unlock them – the OS will handle it when your process exits. The rule is you only free/unlock what *you* allocate/lock. In general, do not interfere with Workbench's own housekeeping.

- That your GUI tool can violate user expectations on the Workbench screen. For instance, all windows on Workbench should obey certain standards: they should have a functioning close gadget (unless they are intentionally persistent utilities), and should honor Workbench preferences (like screen font, colors). Don't assume you can open a borderless window on Workbench screen covering the whole screen – Workbench may consider that rude, and indeed, a borderless backdrop window could conceivably interfere with Workbench's desktop (icons). If you really need a backdrop, consider opening your own screen. Also, if your program modifies Workbench data (like .info files, or window positions), follow documented methods. For example, to tell Workbench to reread icons, send a `WBMessage` or use the AppWindow mechanism rather than poking into Workbench's memory. Essentially **do not assume** you can reach into Workbench internals – always use official APIs.

- That Workbench is always running. While 99% of the time on AmigaOS 3.x the Workbench screen is open, the system *does* allow an expert user to close Workbench (by ending the Workbench task or booting without WB). If your program is a pure GUI tool, it likely won't even be run in that scenario, but if it might be, you should handle the case of `LockPubScreen("Workbench")` failing (returning NULL). This could happen if Workbench isn't running. In such a case, you must either open your own screen or fail gracefully with an error. It's undefined to assume a public screen will be there. Similarly, on systems with multiple public screens, `LockPubScreen(NULL)` locks the default, which is usually Workbench but not always (there's a user preference for default public screen). If you specifically need Workbench, name it. Just be mindful that Workbench might not exist; it's a rare edge case on classic 3.x, but a possible one.

- Finally, do not assume Workbench will clean up after your program. If your program crashes or hangs, Workbench may remain in an inconsistent state (for example, if you left an AppIcon, it might linger). Always clean up (remove AppIcons/AppWindows, free memory, etc.) on exit. Workbench expects well-behaved programs – for instance, Workbench itself will not forcibly remove windows you left open. It's the developer's job to ensure a smooth integration by adhering to these contracts.

Part B – Landmines, Bugs, and Quirks in 3.0/3.1

Even within the bounds of official behavior, there are some known pitfalls and historical bugs in AmigaOS 3.0 and 3.1. This section highlights issues and safe practices for Intuition, GadTools, menus, and ASL on these versions:

- **Closing Windows and Message Ports** (All 3.x versions):
  Problem: If you call `CloseWindow()` on a window while you still have pending messages for it (for example, inside your IDCMP loop before `GetMsg()` has returned NULL), you risk using a freed message port on the next iteration. This often manifests as random crashes or Guru Meditations when exiting the event loop.
  Affected: AmigaOS 2.0 through 3.1 (and beyond) – it's a logic bug in the program, not the OS per se, but very common.
  Safe Pattern: Instead of closing the window immediately when you see an IDCMP_CLOSEWINDOW, set a flag (`done = TRUE`) and break out of the message loop **after** replying to that message. Then close the window outside the loop. This ensures you don't call `GetMsg()` on an invalid port [53]. In summary: handle the close event by signaling your main loop to terminate, not by directly closing the window while still in the loop. This avoids dereferencing stale pointers.

- **Forgetting to Clear MenuStrip** (All 3.x):
  Problem: Freeing or altering a menu that is still attached to an open window can cause memory leaks or crashes. Intuition attaches the menu strip to the window/screen, and if the window is closed or dies while a menu is attached, the OS might not properly clean it up (older Intuition versions leaked memory in this case).
  Affected: OS 2.x, 3.0, 3.1 – the requirement to manually clear menus is documented in RKMs (though OS 3.1 likely frees the menu list on window close, it's not guaranteed in all cases).
  Solution: Always call `ClearMenuStrip(window)` **before** freeing a menu or before closing a window that has a menu [33]. This detaches the menus from Intuition. After that, use `FreeMenus(menuStrip)` to release the menu data. Following this order prevents any OS bug from biting you and ensures no menu memory is "left behind" allocated. It's also good practice to check `ItemAddress` or `item->NextSelect` logic carefully – not clearing `NextSelect` (by processing all selected items) isn't exactly a memory bug, but can yield logical errors.

- **GadTools ListView update quirk** (OS 2.0+ including 3.0/3.1):
  Problem: If you modify the contents of a ListView's label list on the fly (e.g., add or remove entries in the underlying `struct List`) without detaching it first, the program can crash. This is because GadTools might be walking the list at the same time (for rendering or hit-testing). The traversal could happen on an input task or during a refresh.
  Affected: V36 (2.0) through V40 (3.1) – the behavior is inherent in GadTools design (multi-task traversal) [29].
  Workaround: Use the "detach/attach" technique. Call `GT_SetGadgetAttrs(myListViewGadget, window, NULL, GTLV_Labels, ~0, TAG_END)` to detach the list [29], then safely modify your Exec List (e.g., using `AddTail()` / `Remove()` on nodes), and finally call `GT_SetGadgetAttrs()` to attach the new list (with `GTLV_Labels, (ULONG)myList`) and optionally update `GTLV_Top` or other attributes. This ensures GadTools isn't iterating your list while you change it. Always detach before major modifications. If you only need to change one item's text, it's still safer to remove and reinsert that node or detach, update, reattach. This pattern avoids crashes and weird behavior on all 3.x systems.

- **String Gadget Enter vs. Click Focus** (Intuition quirk):
  Issue: GadTools (and Intuition) string gadgets do **not** send an IDCMP event when the user simply clicks out of the field (defocusing it) without pressing Enter. A novice might assume they'll get an update event on every change or when the field loses focus, but in 3.0/3.1 they won't [22]. This can be considered a "quirk" – the user could type text and click a button, and if you only update your internal value on `IDCMP_GADGETUP` from the string, you might still have the old value.
  Affected: All versions up to 3.1 (later versions or custom gadget frameworks like MUI/Reaction have options for immediate updates, but GadTools doesn't aside from manual tricks).
  Solution: Always retrieve the latest string from the gadget's buffer when you need it, rather than relying solely on an event. For example, just before processing an "OK" button, call `GetGadgetAttrs()` or inspect `((struct StringInfo*)stringGadget->SpecialInfo)->Buffer` to get the current text. Alternatively, you can set the string gadget with the `GA_Immediate` flag (officially supported by passing the tag at creation [54]), which causes an `IDCMP_GADGETDOWN` when the gadget is first activated. Even with GA_Immediate, you still won't get an event on every keystroke – it's not like a modern "on change" event. So the safest practice: whenever you need the string's content (like when a user clicks an action button), fetch it directly from the gadget. This avoids missing any final edits.

- **Menu VERIFY deadlock** (All 3.x, design pitfall):
  Issue: The `IDCMP_MENUVERIFY` message is used when an app wants to intercept a menu pick *before* Intuition closes the menu UI, usually to prompt "Are you sure?" or to handle situations like needing to insert a disk, etc. When Intuition sends an `IDCMP_MENUVERIFY`, it effectively **pauses the menu system** until you reply to that message. If your code opens any requester or performs any action that waits for user input *while handling a MENUVERIFY and without replying*, the system will deadlock – Intuition is waiting on your reply, and your code is waiting on Intuition (circular wait) [55].
  Affected: All Intuition versions that support MENUVERIFY (V36+). It's not a bug but a common landmine – especially on OS 3.x where multitasking might obscure the cause.
  Guideline: If you use `MENUVERIFY`, make sure to ReplyMsg() to that IntuiMessage quickly, *before* you do any lengthy operation or open any modal UI (like an `AutoRequest()` or ASL requester). Typically, one uses MENUVERIFY to either refuse an action (by not calling `ReplyMsg` immediately but rather after some condition) or to perform special handling. The key is to not call anything that waits for another Intuition event until you've replied to the verify. If you need to prompt the user (e.g., "Save changes before Quit?" on selecting Quit menu), one safe pattern is: cancel the menu selection (don't reply positively), close the menu (maybe by synthetic cancel or by letting it time out), then open your yes/no requester. Or simply avoid MENUVERIFY and handle such logic after the fact (when the menu pick arrives normally). In short, **beware** that verify messages stall Intuition's input loop – handle them and respond quickly to avoid freezing all windows.

- **Console output and WB programs** (OS 3.0/3.1 quirk):
  Problem: As mentioned in integration, writing to `stdout`/`stderr` when launched from Workbench can cause your program to hang or crash. On OS3.x, if no console is attached and you attempt I/O, the DOS calls may return error or fill an internal buffer until something breaks. Some compilers' startup code open a pseudo-console like `NIL:` for Workbench tasks, but not all. This isn't an Intuition bug but a developer pitfall.
  Affected: Workbench-launched programs on all versions (1.x-3.x). In OS3.1, for example, if you do a `printf` without a console, you might get a Guru Meditation due to a bad file handle.
  Solution: Either avoid console I/O in GUI programs (preferred), or open a console window yourself if needed (using `Open()` or `Execute("NEWSHELL",...)` etc.). At minimum, ensure

your compiled program has sufficient stack and doesn't call exit in a way that skips cleanup (use `exit()` not raw `Exit()` as noted above). Modern practice: use proper GUI dialogs or log to a file for messages, instead of printf in a WB GUI. This prevents the hang described in troubleshooting guides where a Workbench program crashes on its first DOS call due to missing WBStartup reply or output issue [50].

- **Exec Cleanup on Workbench Exit** (OS 3.0 bug fixed in 3.1):
  Issue: There was a small bug in Kickstart 3.0 (V39) where certain resources weren't freed on process termination from Workbench. One example: programs that called `Exit(0)` instead of `exit()` could remain resident. This was largely fixed in 3.1, which properly unloads tasks, but if you ever see that your program's task doesn't disappear from the task list after quitting, it's likely because it bypassed C cleanup.
  Affected: Kickstart 39 (WB 3.0) mainly.
  Workaround: Always pair allocations with frees, and let the C runtime do its job. If you suspect an OS bug (e.g., some intuition structures not freed due to an obscure Intuition bug), you can test on 3.1 which had many bugfixes. Generally, OS3.1 is far more stable – it's recommended to target 3.1 for release even if you maintain 3.0 compatibility, because many quirks (especially in Workbench and scsi.device) were fixed. For instance, the infamous >4GB file bug in scsi.device was a 3.0 issue not relevant to GUI but indicative of 3.0's unfinished state [56].

- **BOOPSI Image Menus Crash** (OS3.1):
  Problem: As noted above, using BOOPSI GUI images in menu items causes a crash in Intuition's LayoutMenus on OS3.1 [38]. The documentation was misleading – OS3.1 cannot actually handle it, despite advertising it.
  Affected: Kickstart 3.1 (V40.4 Intuition). (3.0 didn't advertise the feature at all, so presumably devs wouldn't try there; 3.1 advertised but broken; OS4 fixed it long after.)
  Workaround: Don't use `IM_ITEM` / `IM_SUB` with custom images on these systems. If you need a menu item with an icon or image, a common workaround in classic Amiga is to use the high-bit trick on the menu label to indicate a checkmark or special symbol, or draw into the menu strip after it's rendered (not recommended). The safest is to avoid image menus entirely on 3.x and use text. If you absolutely must, you could create a custom Intuition menu (bypassing GadTools) with your own `MenuItem.Image`, but then you must handle a lot manually – not worth the risk on 3.x.

- **Miscellaneous quirks:**
  There are other small issues – for example, on some early 3.0 systems, the GadTools checkbox gadget's default size constants (CHECKBOXWIDTH/HEIGHT) were not scaled to the screen font, resulting in cut-off checkmarks if the screen font was larger than 8px. OS3.1's gadtools.library fixed some scaling issues (introducing `GTCB_Scaled` tag). If targeting 3.0, be mindful that gadgets might need explicit sizing. Another: under low memory, `CreateMenus` can fail and return NULL – always check for NULL and handle gracefully (don't call LayoutMenus on a NULL menu pointer). If you use ASL requesters, note that asl.library in 3.0/3.1 does not automatically fall back to defaults if you pass illegal combinations of tags; e.g. if you set both `ASL_File` and `ASL_Dir` in a FileRequester (one expects a file name, one a dir name), it might ignore one – provide sensible inputs. Lastly, Intuition's rendering of disabled menu items in 3.0 had a bug where the text was sometimes not ghosted properly (hardly noticeable, but fixed in 3.1). These are minor, but the general rule: test your GUI on both 3.0 and 3.1 if you intend to support both, as 3.1 fixed numerous subtle bugs present in 3.0.

Part C – Canonical Idioms and Patterns (AmigaOS 3.x GUI)

Below are concise, idiomatic code patterns for common GUI tasks on AmigaOS 3.0/3.1. Each is crafted for safety on real hardware and follows best practices. **Note:** Code is C (compatible with VBCC, SAS/C, DICE, etc.). Comments highlight any VBCC specifics or legacy differences. All examples assume the use of <proto/> includes or `-lauto` (auto library open), so function calls like `OpenWindowTags()` and GadTools functions are directly available. Adjust if your environment differs.

1. Window Opening and Event Loop Skeleton

This example opens a window on the Workbench screen and enters a basic event loop to handle the close-gadget. It illustrates proper IDCMP usage and resource cleanup:

```
#include <intuition/intuition.h>
#include <proto/intuition.h>
#include <proto/exec.h>

struct Window *win;
BOOL done = FALSE;
ULONG winSignal, sigMask;

/* Open a window on the Workbench screen */
win = OpenWindowTags(NULL,
    WA_Title,        (ULONG)"MyApp Window",
    WA_Width,        300,
    WA_Height,       100,
    WA_Left,         50,
    WA_Top,          50,
    WA_DragBar,      TRUE,
    WA_DepthGadget,  TRUE,
    WA_CloseGadget,  TRUE,
    WA_Activate,     TRUE,
    WA_SmartRefresh, TRUE,           /* minimize redraw area */
    WA_IDCMP,        IDCMP_CLOSEWINDOW | IDCMP_NEWSIZE,
    WA_PubScreenName,(ULONG)"Workbench",   /* use Workbench screen */
    TAG_END);
if (!win) {
    // Handle error (not enough memory or screen not available)
    return ERROR;
}

/* Obtain the signal mask for the window's UserPort */
winSignal = 1UL << win->UserPort->mp_SigBit;
sigMask   = winSignal;  // here we only wait on one signal

while (!done) {
    ULONG signals = Wait(sigMask);
    if (signals & winSignal) {
        struct IntuiMessage *msg;
        /* Handle all messages waiting */
```

```
        while ((msg = (struct IntuiMessage *)GetMsg(win->UserPort)) != NULL)
{
            ULONG class = msg->Class;
            // It's safe to cache fields if needed, before ReplyMsg.
            ReplyMsg((struct Message*)msg);  /* return message ASAP */
            if (class == IDCMP_CLOSEWINDOW) {
                done = TRUE;
            } else if (class == IDCMP_NEWSIZE) {
                // Window resized by user; could adjust gadget layout here
                // (For GadTools auto-layout, often not needed)
            }
            /* Other IDCMP events (IDCMP_GADGETUP, etc.) would be handled
here */
        }
    }
    /* Could handle other signals (timers, etc.) here by adding to sigMask */
}

/* Clean up: close window (will also remove it from screen) */
CloseWindow(win);
```

Notes: This pattern uses a `while(GetMsg())` loop to drain all messages for each signal wake-up, replying to each [8] [9]. The close event sets `done = TRUE` to break out. We included `IDCMP_NEWSIZE` as an example additional event; in a dynamic GUI you might reposition gadgets there. The window is opened with `WA_SmartRefresh` on Workbench – a common choice for mostly-static content windows (Intuition will only repaint newly exposed areas on expose events, not the whole window). We specified `WA_PubScreenName` to ensure it's on Workbench. If Workbench might not exist, you'd handle the NULL return from OpenWindow accordingly (above, we simply return an error). VBCC: linking with `-lauto` auto-opens intuition.library, so no explicit `OpenLibrary("intuition.library", 39)` is shown. If using older compilers without auto-open, ensure Intuition is open before calling (but since Intuition is a resident library, `OpenWindowTags` will still work as the OS autopens Intuition for you at startup – yet proper style is to open it).

This skeleton is minimal; real code might also handle `IDCMP_REFRESHWINDOW` (for `WFLG_NOCAREREFRESH` windows), `IDCMP_MENUVERIFY`, etc., as needed. Also note we didn't include `ConsoleDevice` output – if you want a Shell-like output in the window, you'd open a console on `win->RPort` (beyond scope here).

2. NewLook Menu Creation and IDCMP_MENUPICK Decoding

Below is an idiomatic way to define and set up a menu bar using GadTools `NewMenu` structures, and how to decode a menu selection from IDCMP:

```
#include <libraries/gadtools.h>
#include <proto/gadtools.h>

/* Define the NewMenu array */
enum { MID_ABOUT=1, MID_QUIT, MID_CHECKOPT };  // menu item IDs for user data
struct NewMenu myMenu[] = {
```

```
    { NM_TITLE, "Project",              0, 0, 0, 0 },                // Title
    { NM_ITEM,  "About...",           "?", 0, 0, (APTR)MID_ABOUT },  // "?"
shortcut
    { NM_ITEM,  "Quit",               "Q", 0, 0, (APTR)MID_QUIT },
    { NM_TITLE, "Options",             0, 0, 0, 0 },
    { NM_ITEM,  "Enable Feature",      0, CHECKIT | MENUTOGGLE, 0,
(APTR)MID_CHECKOPT },
    { NM_END, NULL, 0, 0, 0, 0 }  // end of menu
};

struct Menu *menuStrip;
APTR vi;  // VisualInfo
...
/* Get VisualInfo for layout (assuming screen is already open or locked) */
vi = GetVisualInfo(win->WScreen, TAG_END);
if (vi) {
    menuStrip = CreateMenus(myMenu, GTMN_FrontPen, ~0, TAG_END);
    if (menuStrip) {
        if (!LayoutMenus(menuStrip, vi, GTMN_NewLookMenus, TRUE, TAG_END)) {
            // If LayoutMenus fails, free and null out
            FreeMenus(menuStrip);
            menuStrip = NULL;
        }
    }
    /* We can free the VisualInfo now if we won't need to layout menus or
gadgets further */
    FreeVisualInfo(vi);
}
if (menuStrip) SetMenuStrip(win, menuStrip);
```

This sets up two menu titles ("Project" and "Options"). “About...” and “Quit” are items under Project, with shortcut keys `?` (typically mapped to Help key) and `Q` (usually Amiga+Q). Under Options, we have a checkable item "Enable Feature". We mark it with `CHECKIT|MENUTOGGLE` so it behaves like a toggle checkbox in the menu [57]. We stored custom IDs (enums) in `nm_UserData` to identify each action.

Handling the menu selection in the IDCMP loop:

```
if (msg->Class == IDCMP_MENUPICK) {
    WORD code = msg->Code;
    while (code != MENUNULL) {
        struct MenuItem *item = ItemAddress(menuStrip, code);
        if (item) {
            ULONG userData = (ULONG)GTMENUITEM_USERDATA(item);  // or simply
item->UserData
            switch (userData) {
                case MID_ABOUT:
                    OpenAboutWindow();
                    break;
                case MID_QUIT:
                    done = TRUE;  // will cause main loop to exit and window
```

```
to close
                    break;
                case MID_CHECKOPT:
                    if (item->Flags & CHECKED) {
                        // Option is now enabled
                    } else {
                        // Option is now disabled
                    }
                    break;
            }
        }
        code = item->NextSelect;
    }
}
```

Notes: We use `ItemAddress(menuStrip, code)` to get the MenuItem pointer from the IDCMP code. The `NextSelect` field is used to iterate in case multiple items were selected in one go (rarely used in practice except for multiple checkmarks). Here we stored `nm_UserData` as small integers (enums) for simplicity, but we cast through `GTMENUITEM_USERDATA` macro (or directly use the `item->UserData` field; GadTools defines a macro for compatibility). This is the canonical way to decode menu picks [37] – it's cleaner than manually extracting menu number and item number, though both approaches work.

After use, when shutting down the program or if you need to change the menus, remove and free:

```
ClearMenuStrip(win);
FreeMenus(menuStrip);
```

This ensures no memory leaks or dangling pointers in Intuition for the menus [33].

VBCC Note: The above code uses GadTools functions directly. VBCC's `-lauto` will auto-open gadtools.library (requiring Kickstart 37+). On SAS/C, one would either use `gadtools.lib` in link or open it manually. The code is otherwise identical across compilers.

3. GadTools Gadget Setup: LISTVIEW, STRING, CYCLE, CHECKBOX, BUTTON

This snippet demonstrates creating a set of GadTools gadgets of various types and adding them to a window. It uses a VisualInfo for the Workbench screen and shows safe initialization. We'll set up a simple form UI as an example:

```
#include <intuition/intuition.h>
#include <libraries/gadtools.h>
#include <proto/gadtools.h>
#include <exec/memory.h>
#include <proto/exec.h>

struct Gadget *glist = NULL;
APTR vi;
```

```
struct Screen *wbScreen;
struct Gadget *gad;

/* Prepare Workbench screen context */
wbScreen = LockPubScreen("Workbench");
if (!wbScreen) { /* error handling */ }
vi = GetVisualInfo(wbScreen, TAG_END);
if (!vi) { UnlockPubScreen(NULL, wbScreen); /* handle error */ }

/* Create a context for the gadget list */
gad = CreateContext(&glist);

/* 1. Button gadget */
struct NewGadget ng = {0};  // zero-init
ng.ng_LeftEdge = 10; ng.ng_TopEdge = 10;
ng.ng_Width = 80;  ng.ng_Height = 12;
ng.ng_GadgetText = (UBYTE *)"Apply";
ng.ng_VisualInfo = vi;
ng.ng_GadgetID = 1;
gad = CreateGadget(BUTTON_KIND, gad, &ng,
                   GA_Id, 1,         /* ID can also be set here */
                   TAG_END);

/* 2. String gadget (50-char buffer) */
UBYTE *strBuf = AllocMem(51, MEMF_ANY|MEMF_CLEAR);
if (!strBuf) { /* handle memory error */ }
strcpy(strBuf, "Default text");
ng.ng_LeftEdge = 10; ng.ng_TopEdge = 30;
ng.ng_Width = 200; ng.ng_Height = 10;
ng.ng_GadgetText = NULL;           /* no label on gadget itself */
ng.ng_GadgetID = 2;
gad = CreateGadget(STRING_KIND, gad, &ng,
                   GTST_String,   (ULONG)strBuf,
                   GTST_MaxChars, 50,
                   GA_TabCycle,   TRUE,   /* allow Tab to switch fields */
                   TAG_END);

/* 3. Cycle gadget (dropdown list of choices) */
static CONST_STRPTR cycleLabels[] = { "Option 1", "Option 2", "Option 3",
NULL };
ng.ng_LeftEdge = 10; ng.ng_TopEdge = 50;
ng.ng_Width = 100; ng.ng_Height = 10;
ng.ng_GadgetText = (UBYTE*)"Mode:";
ng.ng_GadgetID = 3;
gad = CreateGadget(CYCLE_KIND, gad, &ng,
                   GTCY_Labels,   (ULONG)cycleLabels,
                   GTCY_Active,   0,     /* default to first option */
                   TAG_END);

/* 4. Checkbox gadget */
ng.ng_LeftEdge = 10; ng.ng_TopEdge = 70;
```

```
ng.ng_Width = CHECKBOX_WIDTH;  ng.ng_Height = CHECKBOX_HEIGHT;  /*  CheckBox
size */
ng.ng_GadgetText = (UBYTE*)"Enable feature";
ng.ng_GadgetID = 4;
gad = CreateGadget(CHECKBOX_KIND, gad, &ng,
                   GTCB_Checked, FALSE,
                   GTCB_Scaled,  TRUE,   /* scale to screen font if needed */
                   TAG_END);

/* 5. ListView gadget */
struct List myList;
NewList(&myList);
/* Populate the list with nodes (each node->ln_Name is displayed) */
struct Node *node;
node = AllocVec(sizeof(struct Node), MEMF_PUBLIC|MEMF_CLEAR);
if(node) { node->ln_Name = "Item A"; AddTail(&myList, node); }
node = AllocVec(sizeof(struct Node), MEMF_PUBLIC|MEMF_CLEAR);
if(node) { node->ln_Name = "Item B"; AddTail(&myList, node); }

ng.ng_LeftEdge = 10; ng.ng_TopEdge = 90;
ng.ng_Width = 100; ng.ng_Height = 50;
ng.ng_GadgetText = NULL;
ng.ng_GadgetID = 5;
gad = CreateGadget(LISTVIEW_KIND, gad, &ng,
                   GTLV_Labels,   (ULONG)&myList,
                   GTLV_Selected, ~0,     /* no selection initially */
                   GTLV_ShowSelected, TRUE,
                   GTLV_ScrollBar,   TRUE,
                   TAG_END);

/* Now open the window with this gadget list */
struct Window *win = OpenWindowTags(NULL,
    WA_Left,        20,
    WA_Top,         30,
    WA_Width,       250,
    WA_Height,      160,
    WA_Title,       (ULONG)"Example GadTools UI",
    WA_DragBar,     TRUE,
    WA_CloseGadget, TRUE,
    WA_DepthGadget, TRUE,
    WA_SizeGadget,  FALSE,
    WA_IDCMP,       IDCMP_CLOSEWINDOW | IDCMP_GADGETUP | IDCMP_GADGETDOWN,
    WA_Gadgets,     (ULONG)glist,
    WA_PubScreen,   (ULONG)wbScreen,
    TAG_END);
```

This creates a window with: a button, a string field, a cycle (drop-down) gadget, a checkbox, and a listview with a scrollbar. Key idioms to note:

- We allocate a buffer for the string gadget text (50 chars + null) and pass it via `GTST_String`. **Important:** GadTools will use this buffer in place (it doesn't allocate its own), so we must keep it

around. We clear it and copy an initial text. The `GTST_MaxChars` ensures the user can't overflow our buffer. We also set `GA_TabCycle` so that Tab key will move focus to the next gadget with `GA_TabCycle` (we set it TRUE for string; GadTools defaults cycles for STR/INT gadgets unless turned off) [58] [22].

- The cycle gadget uses a static array of labels. Because `GTCY_Labels` was introduced in V37, on 3.0 (V39) it's definitely available. We must ensure the array persists (here it's static, so OK). GadTools will display those three options and handle cycling. The first entry (index 0) is active initially.

- The checkbox is straightforward. We used `GTCB_Scaled` to true so that if the screen uses a larger font, the checkbox will scale up from the default 13x8 size [59] [60]. The `GTCB_Checked` FALSE just means it starts unchecked (default anyway). Its GadgetText provides the label next to the box.

- For the listview: we created an Exec list `myList` and added two nodes with `ln_Name` set to item strings. We pass the address of this list in `GTLV_Labels`. GadTools will display "Item A" and "Item B". We use `GTLV_ShowSelected, TRUE` so that the selected line will be highlighted (this tag might default to TRUE on 3.x, but we include it for clarity). `GTLV_Selected, ~0` means no item pre-selected (using ~0 per autodoc to indicate none). If we wanted a pre-selection, we could use an index. Also, `GTLV_ScrollBar, TRUE` ensures a dedicated scroll-bar gadget appears (on 3.x this might be automatic if height is such that not all items fit – but explicit tag is fine).

After opening the window, Intuition will have added and rendered the gadgets. If some gadgets need an initial manual refresh (usually not for GadTools if opened with WA_Gadgets), you could call `RefreshGList(glist, win, NULL, -1)` or `GT_RefreshWindow(win, NULL)` [61] [62], but in practice opening the window with WA_Gadgets draws them.

In the IDCMP loop, when handling `IDCMP_GADGETUP` or `GADGETDOWN`, you can identify which gadget by `IMsg->IAddress` (the gadget pointer) or by the GadgetID we assigned:

```
if (msg->Class == IDCMP_GADGETUP) {
    struct Gadget *gad = (struct Gadget *)msg->IAddress;
    UWORD gid = gad->GadgetID;
    switch (gid) {
      case 1:  /* Apply button clicked */
        DoApply();
        break;
      case 2:  /* String gadget enter pressed (user hit Enter in field) */
        printf("User input: %s\n", ((struct StringInfo*)gad->SpecialInfo)-
>Buffer);
        break;
      case 3:  /* Cycle gadget changed */
        UWORD choice = ((struct Gadget*)gad)->SpecialInfo;  // older
Intuition style
        // Actually, easier: the Code field of IDCMP_GADGETUP for cycles
gives the new index
        choice = msg->Code;
```

```
        printf("Cycle choice now %d\n", choice);
        break;
      case 4:  /* Checkbox toggled */
        BOOL checked = (gad->Flags & GFLG_SELECTED) ? TRUE : FALSE;
        printf("Checkbox state now %d\n", checked);
        break;
      case 5:  /* ListView item selected or double-clicked */
        if (msg->Code == (UWORD)~0) {
            // ~0 code for listview indicates deselection or no selection
        } else {
            UWORD selectedIndex = msg->Code;
            printf("ListView selected index: %u\n", selectedIndex);
            // You can retrieve the node by iterating myList n times or
keeping an array.
        }
        break;
    }
}
```

We use `GadgetID` to differentiate gadgets (the GadgetID corresponds to `ng.ng_GadgetID` we set). Note for cycle gadgets: Intuition sets `msg->Code` to the new active index on IDCMP_GADGETUP, so that's the easiest way to get the selection. For checkboxes, the state can be read from `GFLG_SELECTED` bit or the `msg->Code` may be 0/1 (often 0 for deselected, 1 for selected, though relying on code for boolean isn't officially documented – better to check the flag). Listviews: the `Code` contains the selected item index, or ~0 if none. Also, `IDCMP_GADGETDOWN` can be used for immediate feedback (e.g., a ListView double-click might send a GADGETDOWN followed by an UP with same selection).

Cleanup: When closing the window, free gadgets and other resources:

```
CloseWindow(win);
FreeGadgets(glist);
FreeVisualInfo(vi);
UnlockPubScreen(NULL, wbScreen);
/* Also free the string buffer and list nodes we allocated */
if(strBuf) FreeMem(strBuf, 51);
struct Node *temp;
while ((temp = RemHead(&myList))) {
    FreeVec(temp);
}
```

This order (CloseWindow then FreeGadgets then FreeVisualInfo then UnlockPubScreen) is important [15]. It ensures no gadget is freed while still in use, and the screen lock is released last. We also free our own allocations (string buffer, list nodes). The ListView's attached list was detached automatically by closing the window (Intuition stops referencing it after window gone), but we removed nodes to be safe. If the list were large or dynamic, remember the detach pattern if modifying at runtime.

4. Safe ListView Data and Cleanup Pattern

As a focused idiom, here's how to safely maintain a ListView's list of items:

```
struct List lvList;
NewList(&lvList);

// Function to add an item to ListView safely
void AddListViewItem(struct Window *win, struct Gadget *listViewGadget,
STRPTR text) {
    // Detach list from gadget before modification
    GT_SetGadgetAttrs(listViewGadget, win, NULL,
                      GTLV_Labels, ~0, TAG_END);

    struct Node *node = AllocVec(sizeof(struct Node), MEMF_PUBLIC|
MEMF_CLEAR);
    if (node) {
        node->ln_Name = text;
        AddTail(&lvList, node);
    }
    // Reattach and refresh the gadget
    GT_SetGadgetAttrs(listViewGadget, win, NULL,
                      GTLV_Labels, (ULONG)&lvList,
                      GTLV_Top, 0, /* optionally adjust scroll position */
                      TAG_END);
}
```

This routine detaches the list (by setting `Labels = ~0`), adds a new node, then reattaches. In real use, you might copy the text into a separate allocated buffer and assign `node->ln_Name` to that (to avoid issues if the original `text` goes out of scope). Also, you might want to maintain a maximum length or sort the list, etc.

To remove an item:

```
void RemoveListViewItem(struct Window *win, struct Gadget *listViewGadget,
struct Node *node) {
    GT_SetGadgetAttrs(listViewGadget, win, NULL,
                      GTLV_Labels, ~0, TAG_END);
    if (node) {
        Remove(node);
        FreeVec(node);
    }
    GT_SetGadgetAttrs(listViewGadget, win, NULL,
                      GTLV_Labels, (ULONG)&lvList,
                      GTLV_Top, 0,
                      TAG_END);
}
```

Same pattern: detach, modify, attach. This ensures no concurrent access issues [29]. Always check `node` isn't null, etc. When shutting down the program, if any items remain in the list, it's good to detach (though closing the window detaches implicitly) and free all nodes. As shown above, a while loop removing heads is a quick way to free an Exec list.

One more note: GadTools ListView doesn't automatically free the strings or nodes – they are yours. Also, if you want the ListView selection to persist or detect double-click, you might track `IDCMP_GADGETUP` with `Code == index` and also look for `IDCMP_GADGETDOWN` (which can indicate a double-click when followed quickly by GADGETUP on same item). That's more logic, but important for good UX (many apps interpret double-click on a list as "OK/activate" on that item).

5. Minimal ASL File Requester Pattern (File Picker Dialog)

To integrate an ASL file requester (for example, when the user clicks a "Browse..." button to select a directory or file), use this pattern:

```
#include <libraries/asl.h>
#include <proto/asl.h>
#include <proto/exec.h>

struct FileRequester *fileReq;

/* Open ASL library (if not using -lauto, do this explicitly) */
struct Library *AslBase = OpenLibrary("asl.library", 37);
if (!AslBase) { /* handle error */ }

/* Allocate a file requester object */
fileReq = (struct FileRequester *)AllocAslRequestTags(ASL_FileRequest,
            ASLFR_TitleText,   (ULONG)"Select a file",
            ASLFR_InitialDrawer,(ULONG)"DH0:",      /* start directory */
            ASLFR_InitialFile,  (ULONG)"#?",       /* show all files */
            ASLFR_PositiveText, (ULONG)"OK",
            ASLFR_NegativeText, (ULONG)"Cancel",
            ASLFR_PrivateIDCMP,
TRUE,    /* use our own Window's IDCMP? (here we choose standalone) */
            ASLFR_Screen,
(ULONG)wbScreen,  /* open on Workbench screen */
            TAG_END);
if (!fileReq) { /* handle error (likely out of memory) */ }

/* Display the requester */
if (AslRequestTags(fileReq,
            ASLFR_Window,       (ULONG)win,    /* attach to our window */
            ASLFR_DrawersOnly,
FALSE,         /* FALSE allows file selection */
            TAG_END))
{
    // User picked something
    STRPTR drawer = fileReq->fr_Drawer;
    STRPTR file   = fileReq->fr_File;
```

```
    if (fileReq->fr_Drawer && fileReq->fr_File) {
        UBYTE fullPath[256];
        stccpy(fullPath, drawer, sizeof(fullPath));
        AddPart(fullPath, file, sizeof(fullPath));
        printf("User chose: %s\n", fullPath);
    }
}
else {
    // User hit Cancel or an error occurred
    // (you could check IoErr() for ERROR_BREAK vs ERROR_NO_FREE_STORE, etc.)
}

/* Free the requester */
FreeAslRequest(fileReq);
CloseLibrary(AslBase);
```

Key points in this idiom:

- We open `asl.library` at version 37 (ASL introduced in 2.0). With VBCC `-lauto`, this is automatic; but showing it for completeness.
- We call `AllocAslRequestTags(ASL_FileRequest, ...)`. We supply some useful tags:
- `ASLFR_TitleText` sets the title bar of the requester window.
- `ASLFR_InitialDrawer` and `ASLFR_InitialFile` to set the starting path (`"DH0:"` and `"#?"` meaning show all files). On 3.x, `"#?"` is the wildcard to list all files; we could also use `ASLFR_InitialPattern`.
- `ASLFR_PositiveText` / `NegativeText` to customize the buttons (optional).
- We used `ASLFR_PrivateIDCMP = TRUE` here which means the requester will create its own IDCMP port instead of piggy-backing on our window's port [46]. In practice, it's fine either way; if you use `ASLFR_Window, win` (which we do in AslRequestTags), by default it shares `win`'s IDCMP unless `PrivateIDCMP` is TRUE. Setting it TRUE can avoid any weird interactions with your own IDCMP handling, at the cost of a tiny bit more overhead. Both are safe on 3.x since we call it synchronously.
- `ASLFR_Screen` we set to Workbench screen pointer so it opens on that screen if no window is attached. Since we do attach to `win`, this probably isn't needed; the requester will appear on win's screen. But it doesn't hurt to ensure the screen context.
- We then call `AslRequestTags(fileReq, ...)` to actually open it. Here we pass:
- `ASLFR_Window, win` to anchor it to our window. This usually centers it over the window and greys out input to the window behind (modal behavior).
- `ASLFR_DrawersOnly, FALSE` to allow picking files. If we wanted a directory chooser, we'd set TRUE (then it would only allow selecting drawers, and `fr_File` might be empty with chosen drawer in `fr_Drawer`).
- (We could also use `ASLFR_InitialDrawer` here instead of at Alloc time, to reset path each invocation as needed.)
- If `AslRequestTags` returns TRUE, we access `fileReq->fr_Drawer` and `fileReq->fr_File`. These are C-strings with the selected path and file. We then combine them using `AddPart()` or manual concatenation. We should be careful about buffer size; here `fullPath[256]` is arbitrary, ensure it's large enough for typical paths (255 is a common max for path length on Amiga).
- If it returns FALSE, the user canceled or something failed (like maybe no memory to open the window, etc.). Usually we don't treat cancel as an error, just abort the operation.

- Finally, free the requester with `FreeAslRequest`. This releases its internal data. We also close asl.library (if we opened it). In many cases you might keep asl.library open for the life of your app if you call it often, and only close on exit.

Important: We allocated the FileRequester structure via AllocAslRequest – do not `FreeMem()` or otherwise free `fileReq` yourself. Always use FreeAslRequest.

This pattern can be adapted for font requesters by using `ASL_FontRequest` and FontRequester fields (`fo_FontName`, etc.). The logic is similar: allocate, call AslRequest, check result.

6. Resource Cleanup Order for Amiga GUI Programs

To avoid leaks and ensure stability, free resources in the reverse order you created them. Here's a canonical cleanup sequence for a program that used the above components:

```
// Assume we have: struct Window *win; struct Menu *menuStrip; struct Gadget
*glist;
// Also perhaps fileReq from ASL, and we locked the WB screen.

if (menuStrip) {
    ClearMenuStrip(win);
    FreeMenus(menuStrip);
    menuStrip = NULL;
}

/* Close the window (detach gadgets) */
if (win) {
    CloseWindow(win);
    win = NULL;
}

/* Free GadTools gadgets */
if (glist) {
    FreeGadgets(glist);
    glist = NULL;
}

/* Free any VisualInfo (after gadgets/menus freed) */
if (vi) {
    FreeVisualInfo(vi);
    vi = NULL;
}

/* Unlock public screen if locked */
if (wbScreen) {
    UnlockPubScreen(NULL, wbScreen);
    wbScreen = NULL;
}

/* Free any remaining custom allocations (list nodes, buffers, etc.) */
```

```
while ((node = RemHead(&lvList))) {
    FreeVec(node);
}
if (strBuf) {
    FreeMem(strBuf, strBufSize);
}

/* Close libraries opened (if not using -lauto) */
if (GadToolsBase) CloseLibrary(GadToolsBase);
if (AslBase) CloseLibrary(AslBase);
```

A few explanations:

We remove the menu strip **before** closing the window [33], to be absolutely safe. (Closing the window with an attached menu in 3.1 is usually okay, but doing it explicitly avoids any lingering pointer in the Window structure.) Then we close the window, which also detaches gadgets from Intuition. Then free the gadgets list. We free VisualInfo after the gadgets and menus are gone, because VisualInfo might be providing styling info to them until that point [15]. Unlock the screen last, since we locked it at the very start before opening window/gadgets.

We also demonstrate freeing any custom data (like list nodes we allocated for the ListView, and the string buffer). Those should be freed after the window is closed and gadgets freed, to ensure nothing is using them. In our case, once the window is closed, GadTools no longer needs the string buffer or list, so it's safe to free.

By following this LIFO order of cleanup, you prevent use-after-free and leaks. Notably, *do not* call `CloseLibrary()` on intuition.library or graphics.library; those are usually opened at program start (by OS or by -lauto) and will be closed automatically on exit. GadTools and ASL we closed because we explicitly opened them in this snippet (if using -lauto, don't double-close either).

One more thing: if your program has multiple windows or screens, be sure to handle each's IDCMP and closure independently. Always terminate any background subtasks or cancel any outstanding I/O (like if you started an asynchronous disk IO or an animation) before exit. The mantra is: for every `AllocXXX` or `OpenXXX`, there must be a corresponding Free/Close, and they should be done in an order that doesn't upset dependencies. If in doubt, consult the autodocs for any special cases (e.g., some libraries need to be closed after certain other resources are freed).

By adhering to these patterns, you ensure that your AmigaOS 3.0/3.1 GUI program is robust, clean, and maintains the classic OS expectations. Happy coding!

Sources: Relevant information has been synthesized from the Amiga ROM Kernel Manuals and official developer documentation for Intuition, GadTools, and ASL on OS3.0/3.1 [49] [4] [53] [22] [29] [34] [47] [50] [12] [38], as well as established safe practices from Commodore-era example code [8] [9]. These patterns avoid undefined behaviors and account for known quirks on these OS versions, providing a solid foundation for classic Amiga GUI development.

---

[1] [2] [3] [4] [5] [6] [35] [36] [49] Intuition Library - AmigaOS Documentation Wiki
https://wiki.amigaos.net/wiki/Intuition_Library

7 8 9 34 37 amiga-news.de - Tutorial: Writing an Amiga GUI program in C
https://www.amiga-news.de/en/news/AN-2023-10-00017-EN.html

10 11 12 33 50 51 52 53 55 Troubleshooting Your Software - AmigaOS Documentation Wiki
https://wiki.amigaos.net/wiki/Troubleshooting_Your_Software

13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 31 32 54 58 59 60 61 62 GadTools Gadgets - AmigaOS Documentation Wiki
https://wiki.amigaos.net/wiki/GadTools_Gadgets

28 15 / / The Kinds of GadTools Gadgets / Listview Gadgets
http://amigadev.elowar.com/read/ADCD_2.1/Libraries_Manual_guide/node0267.html

29 30 gadtools.library/GT_SetGadgetAttrsA
https://amigadev.grimore.org/Includes_and_Autodocs_3._guide/node0284.html

38 39 40 Intuition/GadTools image menus (IM_ITEM/IM_SUB) - Hyperion Entertainment Message Boards
https://forum.hyperion-entertainment.com/viewtopic.php?t=94

41 42 43 44 45 46 47 48 ASL Library - AmigaOS Documentation Wiki
https://wiki.amigaos.net/wiki/ASL_Library

56 What's the difference between 3.0 and 3.1 amiga roms? - AmiBay
https://www.amibay.com/threads/whats-the-difference-between-3-0-and-3-1-amiga-roms.55664/

57 Mastering Amiga Programming Secrets
https://oscarbruce.co.uk/amiga/pdf/Mastering_Amiga_Programming_Secrets.pdf